Looking at the terminal of node 2, you can see the confirmation of the exit and removal of the leader and the election of a new leader:

```
Cluster Node [akka://words@127.0.0.1:2552] - Exiting confirmed
➥[akka://words@127.0.0.1:2551]                                          ❶
Cluster Node [akka://words@127.0.0.1:2552] - is the new leader among
➥reachable nodes (more leaders may exist)                               ❷
Cluster Node [akka://words@127.0.0.1:2552] - Leader is removing confirmed
➥Exiting node [akka://words@127.0.0.1:2551]                             ❸
```

❶ Node 2 confirms that node 1 has the Exiting state.

❷ Node 2 becomes the leader.

❸ The leader removes the exiting node.

Gossip protocol

You may wonder how the seed nodes in the example know that the first seed node is leaving, exiting, and finally removed. Akka uses a *gossip* protocol to communicate the state of the cluster to all member nodes of the cluster.

Every node gossips to other nodes about its own state and the states it has seen (the gossip). The protocol makes it possible for all nodes in the cluster to eventually agree about the state of every node. This agreement is called *convergence*, which can occur over time while the nodes gossip to each other. To assist this convergence in the case of unreachable nodes, the Split Brain Resolver (SBR) was created.

A leader for the cluster can be determined after convergence. The first node (in sort order) that is `Up` or `Leaving` automatically becomes the leader. (The full remote address of the node is used to sort nodes, such as akka.tcp://words@ 127.0.0.1:2551.)

Seed nodes 2 and 3 learn that seed node 1 has requested to leave the cluster. As soon as node 1 transitions to the `Exiting` status, seed node 2 automatically takes over and changes the status of the leaving node from `Exiting` to `Removed`. Now the cluster has two